NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

UCS 2312 Data Structures Lab

Assignment 5: QueueADT and its application

Create an ADT for the circular queue data structure using Singly Linked List with the following functions. Each node which consists of Job ID and Burst time.

[CO1, K3]

a. createQueue(Q,size) – initialize size
b. enqueue(Q, data) – enqueue data into the queue
c. dequeue(Q)– returns the element at front
d. isEmpty(Q) – returns 1 if queue empty, otherwise returns 0
e. display(Q) – display the elements in queue

Test the operations of queueADT with the following test cases

Operation	**Expected Output**
dequeue(Q)	Empty
enqueue(Q, J1, 2)	(J1,2)
enqueue(Q, J2, 13)	(J1,2), (J2,13)
enqueue(Q, J3, 5)	(J1,2), (J2,13),(J3,5)
dequeue(Q)	(J1,2)
display(Q)	(J2,13),(J3,5)
dequeue(Q)	(J2,13)
display(Q)	(J3,5)

Best practices to be followed:

- Design before coding
- Usage of algorithm notation
- Use of multi-file C program
- Versioning of code

Application using Queue

1. Job scheduling

   Insert queue with the following contents
   (J1,2), (J2,4), (J3,8), (J4,5), (J5,2), (J6,7), (J7,4), (J8,3) (J9,6) & (J10,6)
   Insert the job into the queue whichever is empty. If it is not empty, insert the job into the queue whichever is having minimum average time
   Display the jobs waiting in both the queues along with their CPU burst time.

2. Build Queue using stacks

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

ALGORITHMS:

1.FUNCTION NAME:create

INPUT:

i)struct queue *

ii)int

OUTPUT: None

1. h->size=n;
2. h->f=h->r=-1;

2.FUNCTION NAME:enqueue

INPUT:

i)struct queue*

ii)int

OUTPUT: None

1. if (isfull(h))
   printf("QUEUE FULL!\n");
2. else
   {
   h->r+=1;
   h->arr[h->r]=ele;
   }

3.FUNCTION NAME:dequeue

INPUT:

i)struct queue*

OUTPUT: None

1. if (isempty(h))
   {
   printf("QUEUE EMPTY!\n");
   h->f=h->r=-1;
   }

2. else
   {
   h->f+=1;
   printf("%d ",h->arr[h->f]);
   }

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

4.FUNCTION NAME:isempty

INPUT:

i)struct queue*

OUTPUT:int

```
1. if (h->f==h->r)
   return 1;
2. else
   return 0;
```

5.FUNCTION NAME:isfull

INPUT:

i)struct queue*

OUTPUT: int

```
1. if (h->r==h->size-1)
      return 1;
2. else
   return 0;
```

6.FUNCTION NAME:display

INPUT:

i)struct queue*

OUTPUT: None

```
1. if isempty(q)
      printf("empty!");
2. else
      {
            for (int i = 0;i<=q->r;i++)
            {
            printf("%d ",q->arr[i]);

            }
       }
```

7.FUNCTION NAME:queue

INPUT:

i)struct stack *

ii)struct stack *

OUTPUT: None

```
struct stack *s;
s=(struct stack *)malloc(sizeof(struct stack));
create(s,5);
push(s,1);
push(s,2);
push(s,3);
```

```
   push(s,4);
   push(s,5);
   printf("--QUEUE WITH STACKS---\n");
   struct stack *s1;
   s1=(struct stack *)malloc(sizeof(struct stack));
   create(s1,5);
   while(!isempty(s))
   {
      int d= pop(s);
      push(s1,d);
   }
   while(!isempty(s1))
   {
      printf("%d ",pop(s1));
   }
```

CODE:
Qadt.h

```
#include <stdio.h>
#include <stdlib.h>

struct queue
{
   int arr[100];
   int f,r;
   int size;
};

struct queue1
{
   int data;
   struct queue1 *next;
};

void init(struct queue *h,int n)
{
   h->size=n;
   h->f=h->r=-1;
}

int isfull(struct queue *h)
{
   if (h->r==h->size-1)
      return 1;
   else
      return 0;
```

```
}

void enqueue(struct queue *h,int ele)
{
   if (isfull(h))
      printf("QUEUE FULL!\n");
   else
   {
      h->r+=1;
      h->arr[h->r]=ele;
   }
}
int isempty(struct queue *h)
{
   if (h->f==h->r)
      return 1;
   else
      return 0;
}
void dequeue(struct queue *h)
{
   if (isempty(h))
   {
      printf("QUEUE EMPTY!\n");
      h->f=h->r=-1;
   }

   else
   {
      h->f+=1;
      printf("%d ",h->arr[h->f]);
   }
}

void enqueue1(struct queue1 *ptr[],int ele)
{
   struct queue1 *temp;
   temp=(struct queue1*)malloc(sizeof(struct queue1));
   temp->data = ele;
   temp->next = NULL;
   if (ptr[0]->next == NULL)
      ptr[0]->next = temp;
   else
      ptr[1]->next = temp;
   ptr[1]=temp;
}
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

```
void dequeue1(struct queue1 *ptr[])
{
   if (ptr[0]->next == NULL)
      printf("QUEUE EMPTY!\n");
   else
   {
      //struct queue1 *temp;
      //temp=ptr[0]->next;
      printf("%d ",ptr[0]->next->data);
      //ptr[0]->next = temp->next;
      ptr[0]->next = ptr[0]->next->next;
   }
}

void display(struct queue*q)
{
   if isempty(q)
      printf("empty!");
   else
   {
      for (int i = 0;i<=q->top;i++)
      {
         printf("%d ",q->arr[i]);

      }
   }
}
```

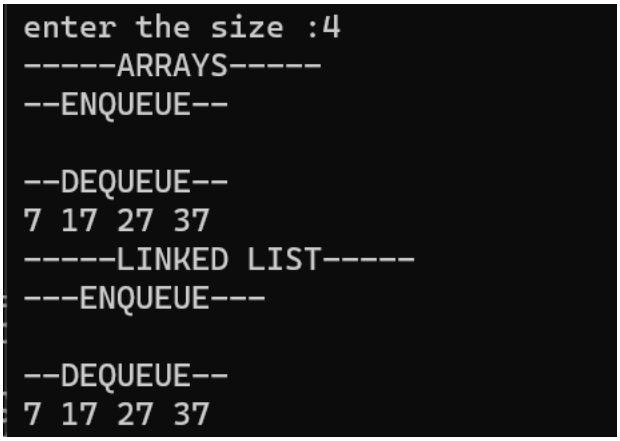
Main.c

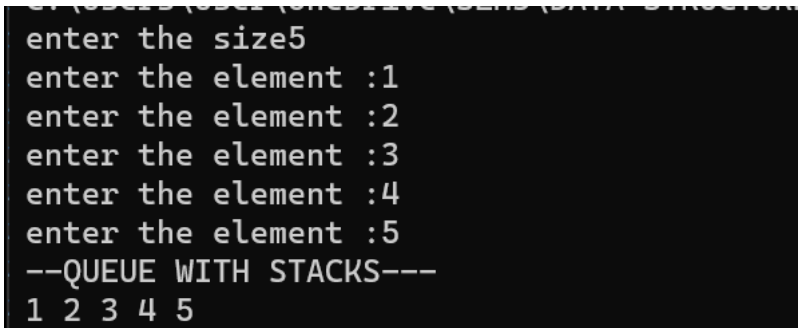
```
#include <stdio.h>
#include <stdlib.h>
#include "qadt.h"
void main()
{
   struct queue *h;
   h=(struct queue*)malloc(sizeof(struct queue));
   printf("enter the size :");
   int n;
   scanf("%d",&n);
   init(h,n);
   printf("-----ARRAYS-----\n");
   printf("--ENQUEUE--\n");
   enqueue(h,7);
   enqueue(h,17);
   enqueue(h,27);
   enqueue(h,37);
```

```
printf("\n--DEQUEUE--\n");
dequeue(h);
dequeue(h);
dequeue(h);
dequeue(h);
printf("\n-----LINKED LIST-----\n");
printf("---ENQUEUE---\n");
struct queue1 *ptr[2];
ptr[0]=(struct queue1*)malloc(sizeof(struct queue1));
ptr[0]->next = NULL;
ptr[1]=(struct queue1*)malloc(sizeof(struct queue1));
ptr[1]->next = NULL;
enqueue1(ptr,7);
enqueue1(ptr,17);
enqueue1(ptr,27);
enqueue1(ptr,37);
printf("\n--DEQUEUE--\n");
dequeue1(ptr);
dequeue1(ptr);
dequeue1(ptr);
dequeue1(ptr);
struct stack *s;
s=(struct stack *)malloc(sizeof(struct stack));
printf("enter the size");
int n;
scanf("%d",&n);
create(s,n);
for (int i = 0;i<n;i++)
{
   printf("enter the element :");
   int a;
   scanf("%d",&a);
   push(s,a);
}
printf("--QUEUE WITH STACKS---\n");
struct stack *s1;
s1=(struct stack *)malloc(sizeof(struct stack));
create(s1,n);
while(!isempty(s))
{
   int d= pop(s);
   push(s1,d);
}
while(!isempty(s1))
{
   printf("%d ",pop(s1));
}
```

}

OUTPUT:





QUESTION 2: Job scheduling

Insert queue with the following contents
(J1,2), (J2,4), (J3,8), (J4,5), (J5,2), (J6,7), (J7,4), (J8,3) (J9,6) & (J10,6)
Insert the job into the queue whichever is empty. If it is not empty, insert the job into the queue whichever is having minimum average time
Display the jobs waiting in both the queues along with their CPU burst time.

CODE:

Timeadt.h

```
#include <stdio.h>
#include <string.h>
struct jobdata
{
   char id[100];
   float time;
};

struct job
{
   struct jobdata arr[100];
   int size;
   int f,r;
};
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

```
void init(struct job *j,int n)
{
   j->size = n;
   j->r=-1;
   j->f=-1;
}

int isfull(struct job *j)
{
   if (j->r==j->size - 1)
      return 1;
   else
      return 0;

}

int isempty(struct job *j)
{
   if (j->r==-1 && j->f==-1)
      return 1;
   else
      return 0;

}

void dequeue(struct job *j)
{
   if (isempty(j))
      printf("EMPTY!\n");
   else
      {
         j->f+=1;
         printf("%s ",j->arr[j->f].id);
         printf(" %f\n",j->arr[j->f].time);
      }

}

void enqueue(struct job *j,struct jobdata *jb)
{
   if (isfull(j))
      printf("QUEUE FULL!\n");
   else
   {
      j->r+=1;
      strcpy((j->arr[j->r]).id ,jb->id);
```

```
      (j->arr[j->r]).time = jb->time;
   }
}

int wt(struct job *j)
{
   if (isempty(j))
      return 0;
   else
   {
      float time = 0;
   for (int i = 0;i<j->size;i++)
   {
      time+=j->arr[i].time;
   }
   return time;
   }
}

int len(struct job *j)
{
   return j->r+1;
}
```

time.c

```
#include <stdio.h>
#include <stdlib.h>
#include "timeadt.h"
#include <string.h>
void main()
{
   struct job *q;
   struct jobdata *q1;
   q=(struct job*)malloc(sizeof(struct job));
   q1=(struct jobdata*)malloc(sizeof(struct jobdata));
   struct job *queue1,*queue2;
   queue1=(struct job*)malloc(sizeof(struct job));
   queue2=(struct job*)malloc(sizeof(struct job));
   printf("enter the number of employees");
   int n;
   scanf("%d",&n);
   init(queue1,n);
   init(queue2,n);
   for (int i = 0;i<n;i++)
   {
      printf("enter the id :");
      char id[100];
```

```
      scanf("%s",id);
      strcpy(q1->id,id);
      printf("enter the time :");
      float time;
      scanf("%f",&time);
      q1->time=time;
      if (wt(queue1)>=wt(queue2))
      {
         enqueue(queue2,q1);
         printf("\nenqueued to queue 2 \n");
         printf("current waiting time of queue1: %d\n",wt(queue1));
         printf("current waiting time of queue2: %d\n\n",wt(queue2));
      }

      else
         {
         enqueue(queue1,q1);
         printf("\nenqueued to queue 1 \n");
         printf("current waiting time of queue1: %d\n",wt(queue1));
         printf("current waiting time of queue2: %d\n\n",wt(queue2));
         }

   }
   printf("--QUEUE 1 ---\n");
   for (int i = 0;i<len(queue1);i++)
   { dequeue(queue1);
   }
   printf("--QUEUE 2 --\n");
   for (int i = 0;i<len(queue2);i++)
   { dequeue(queue2);
   }

}
```

OUTPUT:

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

```
enter the number of employees4
enter the id :j1
enter the time :1

enqueued to queue 2
current waiting time of queue1: 0
current waiting time of queue2: 1

enter the id :j2
enter the time :3

enqueued to queue 1
current waiting time of queue1: 3
current waiting time of queue2: 1

enter the id :j3
enter the time :5

enqueued to queue 2
current waiting time of queue1: 3
current waiting time of queue2: 6

enter the id :j4
enter the time :10

enqueued to queue 1
current waiting time of queue1: 13
current waiting time of queue2: 6

--QUEUE 1 ---
j2  3.000000
j4  10.000000
--QUEUE 2 --
j1  1.000000
j3  5.000000
```